

GestISP

Software de Gestión Integral para ISP

MANUAL TÉCNICO

Arquitectura, decisiones de diseño y guía para colaboradores

Versión del sistema a 9 de agosto de 2026 · 644 pruebas automatizadas

1. Para quién es este documento

Este manual está dirigido a un equipo de desarrollo que va a colaborar en GestISP sin haberlo escrito. No repite lo que se ve leyendo el código: explica la arquitectura, el modelo de datos, las integraciones con equipos de red y —lo más importante— las decisiones de diseño y las trampas conocidas, que es justamente lo que cuesta semanas descubrir por cuenta propia.

Convención del proyecto: Todo el código, los comentarios, los nombres de las pruebas y la interfaz están en español. Es deliberado: quien mantiene el sistema y quien lo usa hablan el mismo idioma. Los nombres de framework y de librerías se dejan como vienen.

1.1 Qué es el sistema

GestISP es una aplicación web monolítica para la gestión integral de un proveedor de internet: CRM, facturación electrónica bajo normativa colombiana, recaudo con caja, inventario, órdenes de trabajo, aprovisionamiento de red sobre equipos Mikrotik y Huawei, y documentación de planta externa de fibra óptica.

1.2 Pila tecnológica

Componente	Versión	Uso
PHP	8.3	Lenguaje. El código usa enums, propiedades promovidas y match.
Laravel	10.48	Framework. Eloquent, colas, tareas programadas, notificaciones.
MySQL	8	Base de datos.
AdminLTE 3.14	jeroennoten/laravel-adminlte	Plantilla del panel. Bootstrap 4.
spatie/laravel-permission	6	Roles y permisos.
maatwebsite/excel	3.1	Exportaciones a Excel.
barryvdh/laravel-dompdf	3.0	Generación de PDF (facturas, recibos, informes).

Componente	Versión	Uso
evilfreelancer/routers-api-php	1.7	API de los routers Mikrotik.
phpseclib/phpseclib	3.0	SSH contra las OLT.
ext-snmp	de PHP	Lectura SNMP de OLT y ONT.
DataTables	1.13.6	Tablas del lado del cliente.
Leaflet + OpenStreetMap	1.9.4	Mapas de la planta externa.
Chart.js	4.4.1	Gráficas de tráfico y potencia.

Sobre el frontend: No hay SPA. Las vistas son Blade con JavaScript puntual. Vite compila solo dos entradas (movimientos de material y proceso de órdenes). El resto de las librerías se cargan por CDN. Es una decisión consciente: el equipo que mantiene esto es pequeño y un frontend independiente duplicaría el coste sin aportar al caso de uso.

2. Arquitectura y convenciones

2.1 Las capas

Capa	Responsabilidad	Regla
Controlador	Validar la petición, resolver permisos y alcance, delegar.	Delgado. Si un método pasa de unas decenas de líneas de lógica, esa lógica va a un servicio.
Servicio (app/Services)	Todo lo que toca más de una tabla o habla con un equipo.	Transaccional y con registro en trazabilidad. Es el único sitio que escribe operaciones compuestas.
Modelo (Eloquent)	Relaciones, scopes, atributos calculados.	Sin lógica de negocio compuesta.
Vista (Blade)	Presentación.	Sin consultas. Los datos llegan precargados del controlador.

2.2 Regla de oro del modelo de datos

No se guarda lo que se puede deducir. Es la regla que sostiene los módulos de red y conviene entenderla antes de tocar nada.

- Un puerto de caja NAP no guarda si está ocupado: se deduce de qué contrato apunta a él.
- Un hilo de fibra no guarda si está en uso: se deduce de sus fusiones, splitters y cajas.
- La ocupación de una caja, de un cable o de una mufla se calcula, nunca se almacena.

El motivo es que un dato duplicado empieza a mentir en cuanto algo se desconecta por otro camino, y un inventario que miente deja de usarse en meses. Solo se guarda lo que NO se puede deducir: dañado, reservado, y los datos que escribe una persona (splitter, zona, descripción).

La excepción, y por qué: El estado de conexión de una cuenta PPPoE Sí se guarda duplicado en `pppoe_accounts` (`connected`, `last_seen_at`, `last_address`, `last_polled_at`). Deducirlo del historial costaba una subconsulta correlacionada por cuenta sobre millones de filas y la pantalla tardaba 20 segundos. La diferencia con los otros casos: aquí hay una única fuente de

verdad y un único proceso que escribe —el muestreador—, mientras que los otros se deducen de tablas que cambian por muchos caminos.

2.3 Alcance por sucursal

Casi todas las tablas de negocio llevan `branch_id`, y la sucursal activa vive en la sesión bajo la clave `branch_id`. El middleware `EnsureBranchSession` la exige.

Los modelos del módulo de red no llevan `branch_id` propio cuando lo heredan: una caja NAP toma la sucursal de su red óptica, una mufla también. El `scope deSucursal()` lo resuelve con `whereHas`.

Trampa conocida: Mezclar `where` y `orWhere` en la misma cadena rompe el filtro de sucursal, porque el OR se sale del grupo. Siempre agrupar: `->where(fn($q) => $q->whereNull("a")->orWhereNull("b"))`. Hubo un caso real de esto en el conteo de cajas sin ubicar.

2.4 Permisos

Se aplican con el middleware `check.permission:NOMBRE` en el constructor de cada controlador. El comando `permissions:sync` recorre los controladores, encuentra los permisos declarados y crea los que falten en la base.

Importante para las pruebas: Un permiso nuevo hay que declararlo TAMBIÉN en `database/seeder/RoleSeeder.php`, o las pruebas darán 403: el seeder es lo que corre en el entorno de pruebas, no `permissions:sync`.

El proyecto NO define un `Gate::before`. Eso significa que la directiva `@can` de Blade evalúa los roles GLOBALES del usuario, no el rol de la sucursal activa, y los dos pueden divergir. El menú se filtra con `RoleBasedMenuFilter`, que sí usa el rol de la sesión. Al añadir un botón, la referencia de verdad es el middleware del controlador destino.

2.5 Trazabilidad

Hay dos mecanismos complementarios:

- Automático: listeners comodín de Eloquent capturan `created`, `updated` y `deleted` de los modelos auditables.
- Explícito: `AuditLogger::action()` para las acciones que no son un simple guardado — un corte, un cobro, una exportación, un descubrimiento de puertos.

Las categorías canónicas están en `AuditLabels::categorias()`. Las de red son red, facturacion, inventario y contratos. Usar una categoría fuera de ese catálogo hace que el registro no se pueda filtrar en la pantalla de trazabilidad.

La telemetría automática (los sondeos que corren cada cinco minutos) se excluye a propósito: llenaría la bitácora sin contar nada que hiciera una persona.

3. Modelo de datos

Unas noventa migraciones. Lo que sigue agrupa las tablas por dominio; los detalles de cada columna están documentados en la propia migración, que en este proyecto lleva comentarios explicando el porqué.

3.1 Núcleo

Tabla	Contenido
branches	Sucursales. Incluye el prefijo y el consecutivo de numeración de contratos.
users, user_branch	Usuarios y a qué sucursales pertenecen, con el rol en cada una.
clients	Clientes. Varios campos son opcionales para permitir altas rápidas.
services, plans, plan_service	Catálogo comercial.
contracts	El centro del sistema. Número consecutivo por sucursal, datos técnicos, ubicación, nap_port_id.
contract_comments	Notas internas del contrato.

3.2 Facturación y recaudo

Tabla	Contenido
invoices	Facturas, con su ciclo de vida y su periodo (índice único por contrato y periodo).
invoice_items, additional_charges	Detalle y cargos fuera del plan, con opción de cuotas.
billing_runs	Cada corrida de facturación masiva.
invoice_numbering_sequences, branch_billing_settings	Rangos y consecutivos DIAN por sucursal.
credit_debit_notes	Notas crédito y débito con su propia numeración.
account_credits	Saldo a favor del cliente (excedentes de notas y anticipos).

Tabla	Contenido
payments, payment_batches, payment_retentions	Cobros, cobros múltiples y retenciones aplicadas.
cash_registers, cash_register_transactions	Cajas y sus movimientos.

Dos vocabularios en contracts.status: El proceso de facturación y el formulario manual escriben estados con nombres distintos. Usar siempre ContractStatusMap para compararlos; comparar la cadena a mano produce errores silenciosos.

3.3 Inventario y trabajo de campo

Tabla	Contenido
warehouses, categories, materials, inventories	Almacén. Categorías y materiales son POR SUCURSAL.
material_movements	Entradas, salidas y traslados, con motivo y responsable.
technical_orders	Órdenes de trabajo, con firma del cliente e imágenes.
technical_order_materials	Material descontado al cerrar una orden.
technical_order_verifications	Verificación posterior de la orden.

3.4 Aprovisionamiento

Tabla	Contenido
routers	Mikrotik con credenciales de API.
pppoe_accounts	Cuentas PPPoE. Incluye el estado de sesión denormalizado.
pppoe_session_metrics	Historial del muestreador (tráfico y conexión).
olts, line_profiles, srv_profiles, vlan_olts	OLT y su configuración leída del equipo.
onts	ONT con su posición, serial, potencia y estado.
ont_metrics	Historial de potencia y tráfico por ONT.
ont_import_runs	Importaciones masivas desde la OLT.

Tabla	Contenido
olt_boards, olt_uplinks, olt_port_metrics	Hardware descubierto por SNMP y su tráfico.

Columnas de texto que engañan: onts.rx_power y onts.status son VARCHAR, no numéricos. Un orderBy directo ordena como texto y "-15.0" queda antes que "-28.5", justo al revés de lo real. Hay que hacer CAST: orderByRaw("CAST(rx_power AS DECIMAL(10,3)) ASC").

3.5 Planta externa (ODN y fibra)

Tabla	Contenido
optical_networks	Red óptica de una sucursal, con consecutivo propio de cajas.
network_zones	Agrupación de puertos PON y cajas para planear.
pon_ports	Puertos PON. Documentación + lo descubierto por SNMP. optical_network_id es NULLABLE.
nap_boxes, nap_ports	Cajas NAP y sus puertos. contracts.nap_port_id es único.
splice_closures	Muflas, con bandejas y capacidad de fusiones.
fiber_cables	Tramos de cable. Extremos POLIMÓRFICOS (Olt, SpliceClosure, NapBox).
cable_strands	Un hilo = una fila, con su buffer y sus colores de norma.
splices	Fusiones. Par ordenado (id menor primero) con índice único.
splitters, splitter_outputs	Splitters dentro de muflas y sus salidas.

3.6 Sistema

Tabla	Contenido
audits	Trazabilidad global.
user_sessions, failed_logins	Sesiones y accesos fallidos.
notifications	Notificaciones enviadas.
permissions, roles y sus pivotes	spatie/laravel-permission.

4. Integraciones con equipos de red

4.1 Mikrotik (MikrotikApiService)

Se conecta por la API nativa de RouterOS. Se usa para crear, editar, habilitar y deshabilitar cuentas PPPoE, cerrar sesiones activas y leer los contadores de tráfico de las interfaces dinámicas.

Una sola petición por router devuelve los contadores de todas las sesiones, que es lo que permite muestrear cientos de cuentas sin coste apreciable.

4.2 OLT por SSH (OltSshService)

phpseclib con un diálogo adaptativo: en vez de asumir el formato de un firmware concreto, lee hasta encontrar el prompt y resuelve solo la paginación y las confirmaciones. Está verificado contra dos estilos distintos de firmware Huawei.

Se usa exclusivamente para ESCRITURAS y consultas puntuales: autorizar una ONT, moverla, deshabilitarla, reiniciarla, CATV. Las lecturas masivas van por SNMP, que es entre 20 y 100 veces más rápido.

Seguridad de comandos: Toda descripción que se manda a la OLT pasa por `OltSshService::descripcionSegura()`, que quita comillas, barras y caracteres de control y trunca a 64. El comando se arma como desc "texto": una comilla sin filtrar cerraría la cadena y lo de atrás lo ejecutaría el equipo.

4.3 OLT y ONT por SNMP (OltSnmpService)

Los OID están en `config/olt_snmp.php`, no en el código, para poder ajustarlos al firmware sin tocar la aplicación. El archivo distingue explícitamente lo verificado de lo que no.

Lo verificado contra equipo real

- Métricas ópticas de ONT: rx_power, tx_power, temperatura, voltaje, corriente, distancia y estado.
- La potencia que la OLT recibe llega DESPLAZADA: el equipo entrega (dBm x 100) + 10000 para evitar negativos.

- En la MIB del equipo probado, la columna .5 es el VOLTAJE (mV) y la .2 la CORRIENTE (mA), al revés de lo que sugiere el orden habitual.
- Historial de conexión: last up, last down y causa de la caída, con cuatro códigos comprobados uno a uno.
- Inventario de ONT: serial en binario con formato Huawei, descripción, perfiles.

Lo NO verificado

La potencia de transmisión del puerto PON y el nombre de la tarjeta salen de OID propietarios que no se han podido comprobar contra el equipo. Están marcados como tales en el config y son opcionales: si no responden, el descubrimiento termina igual y la pantalla muestra un guion. El comando `olt:probe-ports` los contrasta contra un equipo concreto.

4.4 Descubrimiento de hardware (OltHardwareDiscovery)

Recorre `ifDescr` y reconoce puertos PON y uplinks por su nombre. Está construido sobre la IF-MIB estándar (RFC 2863), no sobre MIB propietarias, para funcionar en cualquier equipo con SNMP.

Dos lecciones de equipos reales: Los patrones NO se anclan con `^` sino con `(?:^|^-)`: las MA5800 prefijan cada interfaz con "Huawei-MA5800-V100R018-". Y la posición se busca en `ifDescr`, `ifName` y `ifAlias`, porque hay firmwares (MA5600 V800R015) cuyo `ifDescr` es genérico por tipo —el mismo texto para los dieciséis puertos— y solo `ifName` trae el `frame/slot/port`.

El emparejamiento de un puerto es por posición física (`frame/slot/port`), nunca por `ifIndex`: la OLT reasigna el `ifIndex` al reiniciar una tarjeta. Redescubrir no pisa la documentación (zona, splitter, descripción). Un puerto PON que desaparece no se borra —pueden colgar cajas—; un uplink sí.

4.5 Cómo probar el SNMP sin equipo

`SnmpClient::forOlt()` es estático y no se puede sustituir en una prueba. Para eso existe `SnmpClientFactory`, una envoltura de una línea que sí se puede registrar en el contenedor con un doble. Todo código nuevo que hable SNMP debe pedir la fábrica por constructor, no llamar al estático.

5. Módulo de red: ODN y planta de fibra

5.1 La jerarquía

```
OLT
└─ puerto PON (pon_ports)
    └─ caja NAP (nap_boxes)
        └─ puerto de caja (nap_ports)
            └─ contrato (contracts.nap_port_id)
```

y en paralelo, la fibra que los une:

```
cable (fiber_cables) — extremos polimórficos — OLT | mufla | caja
└─ hilo (cable_strands)
    ├── fusión en mufla (splices)
    ├── entrada/salida de splitter (splitters, splitter_outputs)
    └─ alimenta una caja (nap_boxes.feed_strand_id)
```

5.2 Un hilo tiene DOS extremos

Es la regla que más cuesta ver y la que rompió el primer diseño. Modelar "en uso" como un interruptor hace que ningún hilo pueda llegar nunca al cliente, porque el caso más común de la red —el splitter reparte y cada salida se va por una fibra hasta su caja— usa los dos extremos del mismo hilo.

CableStrand::conexiones() cuenta fusiones, entrada de splitter, salida de splitter y caja alimentada; se admiten hasta dos. estaLibre() significa cero conexiones (para "¿qué hilos vírgenes quedan?"); estaDisponible() significa menos de dos (para poder conectar).

5.3 Código de colores

App\Support\FiberColors implementa la secuencia TIA-598-C de doce colores. Un solo sitio, a propósito: si el sistema dice azul y en el cable es naranja, el técnico corta el que no era. Pasada la docena la secuencia se repite "con trazador", como hace el fabricante.

5.4 Análisis de impacto (FiberPathTracer)

Responde "si corto esto, ¿a quién dejo sin servicio?". No intenta adivinar la dirección de la señal, que es donde fallan estas herramientas. Lo hace por simulación:

1. Arma en memoria el grafo de hilos conectados (fusiones y splitters).
2. Calcula qué cajas NAP son alcanzables desde las OLT.
3. Quita del grafo el elemento a cortar y recalcula.
4. Lo que dejó de ser alcanzable es lo que se cae.

La ventaja del método: Una ruta redundante sale bien sin escribir una sola línea sobre redundancia: si la caja sigue alcanzable por otro camino, no aparece en el impacto.

Depende de dos datos: que los cables que salen de la OLT tengan `from_type = Olt::class`, y que cada caja tenga anotado `feed_strand_id`. Sin el primero no hay desde dónde empezar; sin el segundo el recorrido se corta justo antes del cliente.

Todo se carga en memoria porque cabe de sobra: una red grande son decenas de miles de hilos y unos miles de fusiones, que en PHP se recorren en milisegundos, mientras que hacerlo con SQL recursivo sería lento y mucho más difícil de leer.

6. Facturación

6.1 Numeración DIAN

Cada sucursal tiene su rango autorizado y su consecutivo en `invoice_numbering_sequences`. La asignación bloquea la fila hasta el commit, igual que la numeración de contratos, para que dos emisiones simultáneas no se lleven el mismo número.

6.2 Corridas

`BillingRunController` lanza la facturación masiva. Cada ejecución queda como una fila en `billing_runs` y las facturas quedan enlazadas con `billing_run_id`, lo que permite el detalle con descargas.

6.3 Saldo y retenciones

`Invoice::getPendingAmount()` es total menos pagos menos retenciones. `totalRetenciones()` excluye las retenciones de pagos borrados lógicamente: si no, una anulación dejaría la factura saldada con dinero que nunca entró.

La retención salda la factura pero NO entra a la caja. El asiento de caja se hace solo por la parte en efectivo. En el informe de facturación la tasa de recaudo se calcula sobre lo cancelado, no sobre lo recibido en efectivo, que es lo correcto contablemente.

6.4 Recibo térmico

`PaymentReceipt` genera un recibo por contrato, todos en el mismo PDF. Dos detalles no obvios de `dompdf`:

- El canvas devuelve tamaño A4 hasta que se llama a `render()`. Por eso el alto se fija dentro de `pdf()`, después de renderizar.
- El alto de página es el MÁXIMO de los recibos, no la suma: cada recibo es su propia página.

7. Notificaciones

Ocho notificaciones sobre siete disparadores, por dos canales: correo y WhatsApp. Todas van en cola, así que ninguna demora una pantalla.

El canal de WhatsApp (`App\Notifications\Channels\WhatsAppChannel`) está diseñado para ser intercambiable y arranca en modo simulado, de forma que se puede desarrollar y probar sin una cuenta de Meta. El comando `whatsapp:test-template` permite validar una plantilla concreta.

Los ocho correos comparten una sola plantilla HTML. Reglas de maquetación de correo: tablas para la estructura y estilos en línea, porque los clientes de correo no aplican CSS externo.

8. Tareas programadas

Definidas en app/Console/Kernel.php. La zona horaria es la de la aplicación (America/Bogota), no la del servidor.

Comando	Frecuencia	Qué hace
onts:poll	Cada 5 min	Potencias, estado y tráfico de todas las ONT.
olt:poll-ports	Cada 5 min	Tráfico de puertos PON y uplinks.
pppoe:poll	Cada 5 min	Sesiones y tráfico PPPoE. Mantiene el estado denormalizado.
olt:discover-ports	Diaria 03:15	Redescubre tarjetas, puertos y uplinks.
onts:poll --resolve-traffic --prune=30	Diaria 03:30	Resuelve ifIndex nuevos y poda historial.
pppoe:poll --prune=30	Diaria 03:45	Poda del historial PPPoE.
olt:poll-ports --prune=30	Diaria 04:00	Poda del historial de puertos.
sessions:sweep	Programada	Cierra sesiones inactivas.
invoices:notify-reminders	Programada	Recordatorios de pago y avisos de vencimiento.

Salida a log: Todas escriben en storage/logs con appendOutputTo y tienen onFailure. Sin eso, una OLT caída o una extensión ausente pasarían inadvertidas: la tarea fallaría en silencio.

8.1 Comandos de diagnóstico

Comando	Para qué
olt:snmp-probe {id}	Contrasta cada OID del config contra una ONT real y muestra crudo y normalizado.
olt:probe-ports {id} --interfaces	Lista las interfaces tal como las nombra el equipo y prueba los OID no verificados.

Comando	Para qué
olt:sync-interfaces	Mapea ifIndex a slot/port y repara ONT sin índice.
permissions:sync	Crea en base los permisos declarados en los controladores.

9. Copias de seguridad

La responsabilidad está repartida a propósito: PHP vuelca la base de datos (DatabaseBackup) y un script de shell —`deploy/backup/gestisp-backup.sh`— empaqueta el código y la configuración del servidor, incluidas las de nginx y Apache.

No hay tabla de copias: el repositorio (BackupRepository) lee el directorio. Así, un archivo borrado a mano no deja un registro fantasma.

El cron de las copias es externo al scheduler de Laravel, y la instalación no lo pisa ni fuerza horarios: si el operador cambió la hora, se respeta.

10. Pruebas

644 pruebas y unas 2.300 aserciones, sobre MySQL con RefreshDatabase. Se ejecutan con php artisan test.

10.1 Qué se prueba y qué no

Las pruebas no persiguen cobertura: persiguen las reglas que, de romperse, producen datos incorrectos sin que nadie se entere. Por ejemplo, que la ocupación de una caja NO se guarde, que el análisis de impacto no incluya cajas alimentadas por otro camino, o que redescubrir puertos no pise la documentación.

10.2 Reglas al escribir pruebas aquí

- El SNMP y el SSH van simulados; ninguna prueba puede depender de un equipo de la red.
- Un permiso nuevo debe estar en RoleSeeder o la prueba dará 403.
- Las pruebas que usan travel() y comprueban tasas deben llamar antes a freezeTime(): si no, se les suma el tiempo real de ejecución y pasan solas pero fallan dentro de la suite.
- No ejecutar dos corridas a la vez: comparten la base gestisp_testing y se truncan mutuamente, produciendo fallos falsos en módulos que nadie tocó.

11. Entorno de desarrollo

11.1 Puesta en marcha

```
git clone <repositorio> && cd gestISP
composer install
npm install && npm run build
cp .env.example .env && php artisan key:generate
# configurar DB_* y crear las bases gestisp y gestisp_testing
php artisan migrate --seed
php artisan permissions:sync
php artisan serve
```

11.2 Requisitos del servidor

- PHP 8.3 con las extensiones habituales de Laravel más ext-snmpp.
- MySQL 8.
- Acceso de red a los routers Mikrotik (API) y a las OLT (SSH y SNMP).
- Un cron que ejecute schedule:run cada minuto.

11.3 Despliegue

```
cd /var/www/gestisp \
&& git pull origin master \
&& php artisan migrate --force \
&& php artisan permissions:sync \
&& php artisan route:clear && php artisan route:cache \
&& php artisan view:clear && php artisan view:cache \
&& php artisan config:cache
```

Encadenar con &&: Es importante. route:cache puede fallar (por ejemplo, por un nombre de ruta duplicado) y sin el encadenado el despliegue continuaría dejando el sistema con la caché de rutas vieja, sin que nadie se entere. Pasó exactamente eso en producción.

npm en el servidor: Si node_modules se copió desde Windows, los binarios pierden el bit de ejecución y npm run build falla con "Permission denied". Se arregla con `find node_modules -type f -path "*/bin/*" -exec chmod +x {} +` o compilando en local y sincronizando public/build (que está en .gitignore).

12. Trampas conocidas

Esta sección es la que más tiempo ahorra. Son errores reales que costaron horas de diagnóstico.

12.1 Blade

- NUNCA mezclar `@php` inline con `@php...@endphp` de bloque en el mismo archivo: el emparejador de bloques crudos toma el inline como apertura y lo casa con el `@endphp` que venga después, tragándose todo lo del medio como PHP. El error señala una línea inocente.
- NUNCA escribir el nombre de una directiva con arroba dentro de un comentario Blade: el emparejador corre antes de quitar los comentarios y la ejecuta igual.
- `@json(...)` con arrays anidados en varias líneas compila partido: Blade corta el argumento en el primer paréntesis que cree de cierre y no cuenta los corchetes. Armar el array en un bloque `@php` y volcarlo con `toJson()`.
- `groupBy()` descarta las claves del array. Si se agrupan columnas indexadas hay que usar `preserveKeys: true`, o los identificadores del HTML se repiten entre grupos.

12.2 PHP y Laravel

- `abort(403)` lanza `HttpException`, que EXTIENDE `RuntimeException`. Un `catch (\RuntimeException)` alrededor de una llamada que aborta convierte el 403 en un redirect y anula el control de acceso. Atrapar antes `HttpExceptionInterface` y relanzar.
- `catch (\Exception)` se traga `ValidationException` y convierte un error de validación en un error genérico. Hay que relanzarla. Es un patrón que apareció en varios controladores.
- Ninguna columna puede llamarse igual que una relación: Eloquent resuelve primero los atributos, así que `$modelo->relacion` devolvería el valor de la columna.
- Un campo que falta en `$fillable` se descarta EN SILENCIO en una asignación masiva. Ha pasado con `if_index` y con `payment_batch_id`.

12.3 Rutas

- Dos rutas con el mismo nombre funcionan en desarrollo (gana la última) pero hacen fallar `route:cache` por completo. El sistema queda entonces con la caché vieja y las rutas nuevas no

existen.

- Las rutas fijas van SIEMPRE antes que las de parámetro: /naps/mapa después de /naps/{nap} haría que Laravel buscara una caja con código "mapa".

12.4 Base de datos

- orWhere sin agrupar rompe los filtros anteriores, incluido el de sucursal.
- Las columnas VARCHAR que contienen números (onts.rx_power) ordenan como texto.
- doctrine/dbal no está instalado, así que ->change() no funciona: los cambios de tipo se hacen con DB::statement y SQL directo.

13. Estado actual y pendientes

13.1 Verificado contra equipos reales

- OID de métricas de ONT y sus escalas, contra dos OLT en producción.
- Diálogo SSH contra dos estilos de firmware Huawei.
- Reconocimiento de interfaces en MA5800 V100R018.

13.2 Pendiente de verificar

- MA5600 V800R015: confirmar que la posición de los puertos llega por ifName; si no, decodificarla desde el ifIndex (hay una fórmula documentada de Huawei que NO se ha comprobado y por eso no está implementada).
- Uplinks en las OLT de producción: confirmar que se reconocen con los patrones actuales.
- OID propietarios de potencia de puerto PON y nombre de tarjeta.

13.3 Deuda técnica conocida

- Queda por hacer un barrido completo del patrón `catch(\Exception)` que se traga `ValidationException`; se corrigió donde se detectó, pero no se ha revisado el proyecto entero.
- Falta un comando para rellenar las credenciales PPPoE de contratos vinculados antes de que `ContractLinker` las copiara.
- El listado de contratos y el de ONT cargan todas las filas y paginan en el cliente; con volúmenes muy altos convendría pasar a procesamiento en servidor.
- Las cajas de empalme cubren muflas y splitters; no se modelan reservas de fibra ni empalmes de derivación en poste sin mufla.